

P2468R0

The Equality Operator You Are Looking For

Published Proposal, 2020-06-18

This version:

<http://wg21.link/p2468r0>

Authors:

[Barry Revzin](#)

[Bjarne Stroustrup](#)

[Cameron DaCamara](#)

[Daveed Vandevoorde](#)

[Gabriel Dos Reis](#)

[Herb Sutter](#)

[Jason Merrill](#)

[Jonathan Caves](#)

[Richard Smith](#)

[Ville Voutilainen](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper details some changes to make rewriting equality in expressions less of a breaking change

Table of Contents

- 1 **Rewritten equality/inequality in expressions**
- 1.1 Compiler disagreements

- 1.2 Existing code impact
 - 1.2.1 Open source sampling
- 1.3 Proposed resolution
 - 1.3.1 First resolution
 - 1.3.2 Second resolution

2 Wording

3 Appendix

- 3.1 Code patterns which fail to compile even under new rules
- 3.2 Code patterns which fail at runtime

§ 1. Rewritten equality/inequality in expressions

§ 1.1. Compiler disagreements

```
struct S {  
    bool operator==(const S&) { return true; } // The non-const member is imp  
    bool operator!=(const S&) { return false; } // The non-const member is in  
};  
bool b = S{} != S{};
```

GCC accepts this (despite overload resolution being ambiguous under C++20).

Clang accepts with a warning indicating the rewrite resulting in ambiguous overload resolution.

```
<source>:5:14: warning: ISO C++20 considers use of overloaded operator '!='  
bool b = S{} != S{};
```

MSVC accepts this by eagerly dropping rewritten candidates.

The reason this sample should result in an ambiguity and does not get to the intended tiebreaker step [\[over.match.general\]/2.8](#) is because ambiguity is actually the result of finding best conversions. During the best conversions step we discover that the candidate `operator!=(const`

`S&)` and rewritten candidate `operator==(const S&)` are equally as good as each other. To the compiler, it is as if the user had written:

```
struct S {
    friend bool f(S, const S&) { ... }
    friend bool f(const S&, S) { ... }
};
bool b = f(S{}, S{});
```

Which is a clear ambiguity during the comparison of best conversions.

§ 1.2. Existing code impact

It is clear to the author(s) of this paper, since the compiler vendors feel the need to bypass the overload resolution rules defined in the standard that there must be some valid code patterns for which the new overload resolution rules unintentionally break.

Using MSVC we implemented the strict rules defined by `[over.match.general]` and ran the compiler against a series of open source projects to gather data on how impactful the strict application of the new overload resolution rules can be.

§ 1.2.1. Open source sampling

The results of applying the strict rules are as follows:

Total Projects	Failed Projects
59	20

Many of the failures are caused by the first code sample mentioned. Other failures include:

```
template <typename T>
struct Base {
    bool operator==(const T&) const;
};

struct Derived : Base<Derived> { };

bool b = Derived{} == Derived{};
```

In this case the user intended to use some common base class to implement all of the comparison operators. Because the new operator rewriting rules will also add the synthesized candidate to the overload set the result becomes ambiguous when trying to compare best conversions. Both the regular candidate and the synthesized candidate contain a derived-to-base conversion which makes one not strictly better than the other.

We feel that the code above should be **accepted**.

```
template <bool>
struct GenericIterator {
    using ConstIterator = GenericIterator<true>;
    using NonConstIterator = GenericIterator<false>;
    GenericIterator() = default;
    GenericIterator(const NonConstIterator&);

    bool operator==(ConstIterator) const;
};
using Iterator = GenericIterator<false>;

bool b = Iterator{} == Iterator{};
```

This is a scenario where the user is depending on implicit conversions to get the desired effect of comparisons being compared as a `ConstIterator`. The issue is that the converting constructor enables conversions on both sides which are no better than the other when considering the reversed candidate.

We feel that the code above should be **accepted**.

```
struct Iterator {
    Iterator();
    Iterator(int*);
    bool operator==(const Iterator&) const;
    operator int*() const;
};

bool b = nullptr != Iterator{};
```

This code was relying on the fact that an implicit conversion gives the user `!=` for free. The issue is that this implicit conversion creates a scenario where the user-defined conversion operator creates a

case where it is not a better choice than the reversed parameter candidate rewrite where a temporary `Iterator` type is constructed from `nullptr`.

We feel that this is a case where C++20 helps the user identify possible semantic problems. It is a case we should **reject** despite being observed in two different projects.

```
using ubool = unsigned char;

struct S {
    operator bool() const;
};
ubool operator==(S, S);

ubool b = S{} != S{};
```

Based on the merge of [P1630R1](#) there is a case added for the rewritten candidate chosen must return cv-`bool` and if it does not the candidate is rejected, but because that condition happens after overload resolution the code author ends up seeing:

```
error C2088: '!=': illegal for struct
```

While this paper does not want to tackle that wording, we feel that the code above should remain **rejected** until a future update.

§ 1.3. Proposed resolution

§ 1.3.1. First resolution

To help address the issues mentioned above we gathered input from the following individuals regarding how GCC and Clang implements overload resolution:

- Jason Merrill
- Richard Smith

The implementation approach taken by GCC appeared to be the most permissive and applied seemingly reasonable rules which allow most of the above samples to compile. The GCC approach is generalized as:

Before comparing candidates for best conversion sequences, compare each candidate to each other candidate (or function template they are specializations of) and if the parameter types match and one is a rewritten candidate the rewritten candidate is not considered for later tiebreakers.

After implementing the rule above in MSVC we obtained the following results from running that compiler against open source projects:

Total Projects	Failed Projects
59	10 (<i>down from 20</i>)

It is immediately clear that the GCC implementation was on the correct path to a good solution. The GCC approach ticked all the boxes for code we wanted to compile (mentioned above) while maintaining the spirit of the original operator rewriting proposal [P0515R3](#) by allowing heterogeneous comparisons.

§ 1.3.2. Second resolution

After discussing the GCC rule Richard Smith proposed a new one:

what if we do not consider rewriting to an `operator==` if there is an `operator!=` with the same signature declared in the same scope? One could argue that all reasonable C++<=17 overload sets will declare `operator==` and `operator!=` together, with the same signatures, and all code intending to use the C++20 rules will declare only `operator==`. Then the way you turn off rewriting is by doing exactly what you did before C++20: you write an `operator!=` with your `operator==`.

Note that the above proposal is nearly identical to the GCC approach with the exception of the 'declared in the same scope' rule. After implementing the above rule in MSVC (but applying to any rewritten operator not just `==`) we obtained the following results:

Total Projects	Failed Projects
59	10

The results are identical to that of the GCC rule which implies that we neither regressed behavior nor improved it. The good thing about the rule as proposed by Richard is that it offers more freedom for the implementation to blindly rewrite `!=` using `operator==`. This proposal still checks all the boxes above of code patterns we want to enable while also maintaining strong rules around comparing viable candidates for rewriting.

Based on the results and the principled approach of the latter rule we propose this resolution for standardization.

§ 2. Wording

Change in 12.2.2.3 [over.match.oper] paragraph 3:

The rewritten candidate set is determined as follows:

- For the relational ([expr.rel]) operators, the rewritten candidates include all non-rewritten candidates for the expression $x \leq y$.
- For the relational ([expr.rel]) and three-way comparison ([expr.spaceship]) operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each non-rewritten candidate for the expression $y \leq x$.
- For the != operator ([expr.eq]), the rewritten candidates include all non-rewritten candidates for the expression $x == y$.
- For the equality operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each non-rewritten candidate for the expression $y == x$.
- For any rewritten operator candidate, if there exists another candidate with the same signature declared in the same scope and that candidate is a non-rewritten candidate then the rewritten candidate is removed from the candidate set.
- For all other operators, the rewritten candidate set is empty.

[Note 2: A candidate synthesized from a member candidate has its implicit object parameter as the second parameter, thus implicit conversions are considered for the first, but not for the second, parameter. — end note]

§ 3. Appendix

§ 3.1. Code patterns which fail to compile even under new rules

```
template<class Derived>
struct Base {
    int operator==(const double&) const;
    friend inline int operator==(const double&, const Derived&);
    int operator!=(const double&) const;
    friend inline int operator!=(const double&, const Derived&);
};

struct X : Base<X> { };

bool b = X{} == 0.;
```

Fails due to requirements of rewritten operator returning cv-**bool**.

```
struct Base {
    bool operator==(const Base&) const;
    bool operator!=(const Base&) const;
};

struct Derived : Base {
    Derived(const Base&);
    bool operator==(const Derived& rhs) const {
        return static_cast<const Base&>(*this) == rhs;
    }
};
```

The code above fails due to relying on a derived-to-base conversion on both sides of the comparison (once the synthesized candidate is taken into account). If one imagines a similar scenario:

```
bool b1 = Derived{} == Base{};
bool b2 = Base{} == Derived{};
```

This would also be ambiguous even without the definition of `Derived::operator==` and in our view is considered a bugfix in C++20 which forces the user to carefully consider how objects are being compared.

§ 3.2. Code patterns which fail at runtime

In C++20, because of how candidate sets are expanded there are a few scenarios where a new candidate introduced is then selected as the best candidate without any code change. In many cases this can be fine but there are a few cases we identified as being potentially problematic for code authors.

```
struct iterator;
struct const_iterator {
    const_iterator(const iterator&);
    bool operator==(const const_iterator &ci) const;
};

struct iterator {
    bool operator==(const const_iterator &ci) const { return ci == *this; }
};
```

In C++17 the sample above would compile and the function selected for the comparison `ci == *this` would be `const_iterator::operator==(const const_iterator&)`. In C++20 the function chosen for the same comparison is the (rewritten) function `iterator::operator==(const const_iterator&)` using the reversed parameter order rule. The chosen function in C++20 causes a new runtime failure where the comparison will cause infinite recursion where there wasn't one before.

Another example:

```
struct U { };

struct S {
    template <typename T>
    friend bool operator==(const S&, const T&) { return true; }

    friend bool operator==(const U& u, const S& s) {
        return s == u;
    }
};

bool b = U{} == S{};
```

In C++17 the user intended for comparisons to be dispatched to the templated `operator==`. In C++20 the templated `operator==` is considered to be a worse match based on [\[over.match.best.general\]/2.4](#) which is a tiebreaker before rewritten candidate tiebreakers, which makes `operator==(const U& u, const S& s)` the best match for the comparison `s == u`.